University of Cape Town
Department of Computer Science

Computer Science CSC3003S
Class Test 2, 2018

Answer all questions.
Approximate marks per question are shown in brackets.

Time: 40 minutes **Marks: 40**

Section 1. Advanced Software Design [7 Marks]

Question 1 [7 Marks]

a. Most Free and Open Source Software projects fail. Give two reasons why this typically happens. [2]

Any two: [1 Mark for each correct. Max 2 Marks]

- *Structural issues*
- *unrealistic or vague specifications*
- *insufficient design*
- *Unrealistic expectations*
- *too few active developers*
- *open code has to be made comprehensible to strangers*
- *needs suitable tools, development web, communication tools*
- *Barriers to entry for new developers and new users*
- *poor presentation, documentation and packaging*
- *Challenges in project management*
- *In-house management practices don't work*

b. Some companies have open source software as part of their business model, over and above possibly using it to run their business operations. Describe two ways companies make money from open source software. [2]

Any two: [1 Mark for each correct. Max 2 Marks]

- *Dual licensing, with free entry level version, and upgraded high level version having a cost*
- *Service contracts featuring support and updates*
- *Paid for training and learning materials*
- *Developing customised extensions*

c. Model-View-Controller (MVC) is a popular design pattern. Describe what each component of MVC does. [3]

Model: defines the data [1]

View: defines the display/user interface [1]

Controller: contains control [1]

Section 2. Theory of Algorithms [24 Marks]

Question 2 [24 Marks]

a. Which **TWO** of the following statements are **TRUE**? [2]

i. $\Theta (n + \log n) = \Theta (n)$

ii. if $x \in O (n \log n)$ then $x \in \Theta (n \log n)$

iii. if $x \in O (n \log n)$ then $x \in O (n^2)$

iv. if $x \in O (n \log n)$ then $x \in O (n)$

i. $\Theta (n + \log n) = \Theta (n)$ [1]

iii. if $x \in O (n \log n)$ then $x \in O (n2)$ [1]

b. Show the different approaches of the algorithmic techniques below, using the example of exponentiation (a^N). Each answer should take one line. [4]

i. Brute force

ii. Divide and conquer

iii. Decrease and conquer

iv. Transform and conquer

i. $a * a * a \ldots * a$ N times [1]

ii. $(a^{N/2}) * (a^{N/2})$ [1]

iii. $(a^{N-1}) * a$ OR $(a^{N/2})^2$ [1]

iv. Horner's rule OR Binary Exponentiation [1]

c. Two other techniques are Greedy Algorithms and Space-Time-Trade-off. Briefly describe these 2 approaches to problem solving (1 line each). [2]

Space-time: use additional data structures/memory/variables/space in order to make a faster solution/algorithm possible [1]

Greedy: make the optimum choice at each step. [1]

d. We have the academic records of all UCT students who have taken any CSC course during the past decade. We wish to sort them by alphabetical order of their Faculty (so all Commerce students first, then all Engineering ones, … finally all Science students). Students from all 6 of UCT's Faculties have taken a CSC course.

i. Outline (do not just name it) a method of solving this problem which is more efficient than quicksort and also uses key comparisons (i.e. do not describe radix sort). [2]

ii. What is the complexity of your more efficient algorithm above? [1]

i. Go through input counting how many from each faculty. [1 mark].

Then set the 6 end-position values for each Faculty accordingly. Finally go through the input again from end to start, using the end-position of their Faculty to put them in the sorted array, and decrement that value to indicate that position has been filled. [1 mark]

ii. linear i.e. O(N). [1]

e. You are searching for the first occurrence of the string REFERER in a text.

i. Show the shift table that Horspool's algorithm would use. [2]

ii. Show the good-suffix shift table that the Boyer-Moore algorithm would use for this search string REFERER. [2]

i.1 mark for characters (E,F,R) 1 mark for values. Elements in any order

E	F	R
1	4	2

ii. 1 mark for indexes (1 to 6). 1 mark for values. Elements in any order.

1	2	3	4	5	6
6	2	6	6	6	6

f. You are given a row of integers. You can choose as many as you like except that you cannot take any 2 integers that are adjacent (next to each other). For example, if the row has values 4, 2, 8, 6 then you can take the 4 only of you don't take the 2, you can take the 2 only if you take neither the 8 nor the 4, … you can take the 6 only if you don't take the 8.

i. State in words or pseudocode how you would you use Dynamic Programming to find the maximum value you can take [2]

ii. Show what the array would contain after solving the example below: [1]
4, 2, 8, 6
Maximum is 12 (take the 4 and the 8).

i. Go from start to end of list, each time choosing max(with new value, without new value) and storing it in the array [1 mark]

For the j'th item, A[j] is set to max(A[j-2]+newval , A[j-1]) [1 mark]

ii. Array [1 mark]

1	2	3	4
4	4	12	12

g. The results of a race are recorded in a file of 100 000 records, where each record contains the position that person came and the time they took, e.g. < 42, 1234> means the person came 42nd and took exactly 1234 seconds to run the race. The file is in order from winner to loser, i.e. in increasing order of position starting at 1.

Consider the problem of knowing the time you took in the race and wanting to look up your position i.e. the input is the number of seconds (e.g. 1234) and the output is the corresponding position (e.g. 42). Assume there were no ties.

Outline briefly in words (about one line each) ***three*** (3) different algorithms you could use to solve this problem and give the complexity of each. [6]

Go through the entire list from start to end which is linear or O(N). *[2 marks]*

Do a binary search which is logarithmic O(log N). *[2 marks]*

Use linear interpolation which is O(log log N). *[2 marks]*

Section 3. Theory of Computation [9 Marks]

Question 3 [9 Marks]

a. Prove, using the formal definitions of Deterministic Finite Automata and Non-Deterministic Finite Automata, that for every Deterministic Finite Automaton there is a Non-Deterministic Finite Automaton that accepts exactly the same language. [4]

Let D = (Q, Sigma, delta, q0, F). We need to find an NFA N such that N accepts the same *language as D.* *[1]*

We define N as: (Q,Sigma,delta', q0, F) where q' is defined as follows: *[1]*

For every q in Q and every s in Sigma, delta'(q,s) = {delta(q,s)} *[1]*

For every q in Q, delta'(q,epsilon) = emptyset *[1]*

b. Suppose we want to prove that for any regular language A, A* is also regular. We know that if A is regular, then there is a Non-Deterministic Finite Automaton N such that N accepts A. Now, from N we construct a new Non-Deterministic Automaton N' as follows: for every accept state s of N, add an epsilon-transition from s to the start state of N (if there isn't one already). Does N' accept A*? If it does, explain why in detail. If it does not, explain why not, indicate what language it accepts, and explain how to modify N' so that it accepts A*. [5]

N' does not always accept A.* *[1]*

If A contains the empty string, then N' accepts A. If not, then N' accepts every element of A* except the empty string.* *[1]*

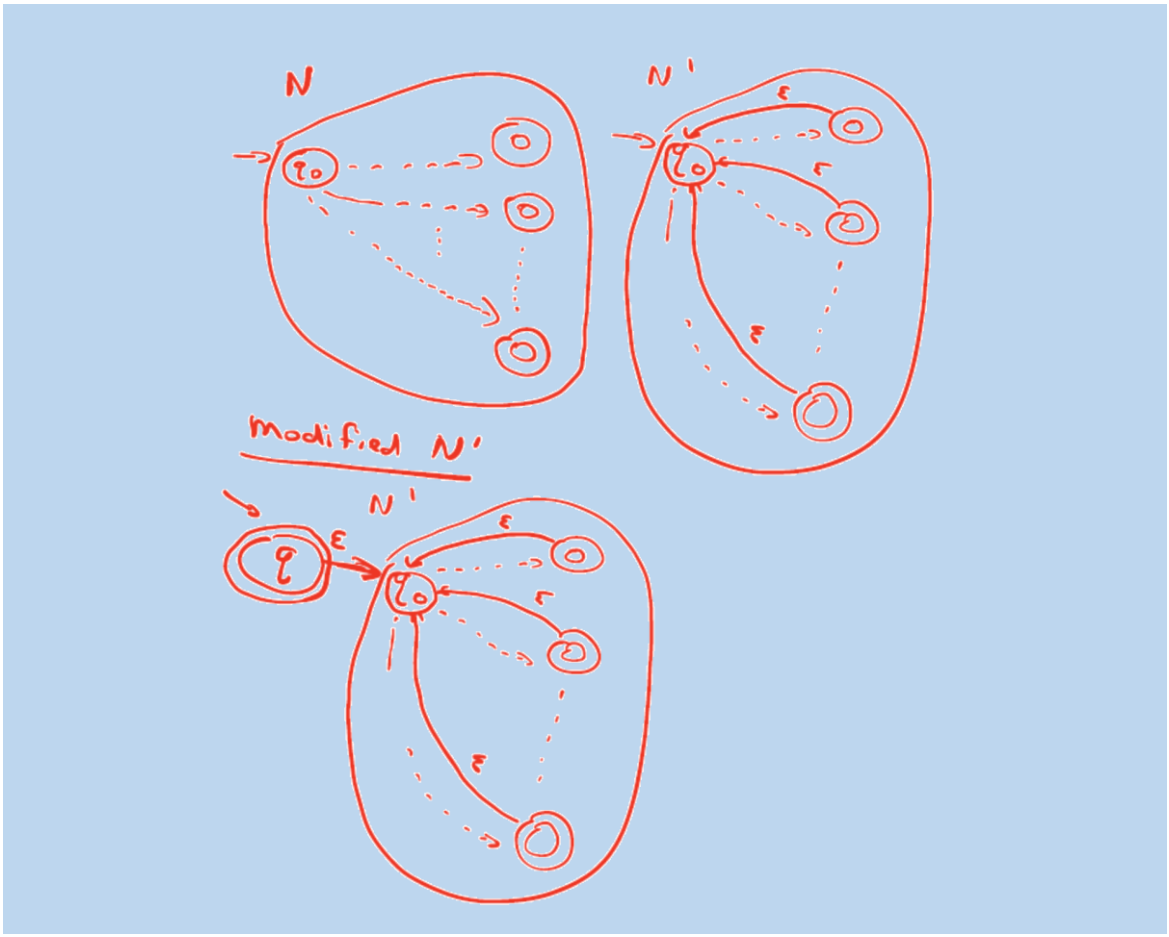
To ensure that N' always accepts A:*

- add a new start state, say q, to N'; *[1]*

- make q an accept state; *[1]*

- add an epsilon-transition from q to the old start state of N'. *[1]*

It's also fine if they draw the NFA N'.


N
N'
ε
ε
ε
Modified N'
N'
q
ε
ε
ε
ε